

Exercitiul 1 — Strategy: schimbi comportamentul, nu clasa

Cerința exercițiului 1 din modulul de design patterns.

Context

Un magazin online calculează **costul de livrare** al unei comenzi. Costul depinde de metoda aleasă: standard, express sau o promoție „livrare gratuită”. Aceeași comandă (aceeași greutate, aceeași distanță) da alt cost, în funcție de metoda.

Prima variantă la care se gândește oricine: o metodă cu un `switch` pe un enum `TipLivrare`. Merge. Problema apare când firma adaugă a patra metodă de livrare — și a cincea. Fix problema pe care ai batut-o la **interfaces ex5 (open/closed)**: ca să adaugi un comportament nou, ești obligat să RE-DESCHIZI și să editezi o metodă care deja mergea.

Strategy e răspunsul: fiecare metodă de calcul devine o clasă separată care semnează același contract. `Comanda` nu mai știe CUM se calculează — ține o strategie și o întreabă. Poți schimba strategia la runtime, iar o metodă nouă = o clasă nouă, fără să atingi nimic din ce merge deja.

Contractul

```
public interface ILivrareStrategie
{
    string Nume { get; }
    decimal CalculeazaCost(decimal greutateKg, decimal distantaKm);
}
```

`Comanda` lucrează doar cu `ILivrareStrategie` — nu-i pasă ce strategie concretă i-ai dat, doar că știe să calculeze un cost.

Strategiile (formulele)

Strategie	Nume	Cost
LivrareStandard	"Standard"	$5 + 0.5 * greutate + 0.1 * distanta$
LivrareExpress	"Express"	$12 + 1.0 * greutate + 0.25 * distanta$
LivrareGratuita	"Gratuita"	0 (promotie — ignora greutatea si distanta)

Pasii (in ordinea asta — primul pas e o capcana intentionata)

- Simte durerea intai.** Pe o foaie (sau intr-o metoda pe care o STERGI dupa), scrie calculul cu `switch (tip)` pe un enum `TipLivrare { Standard, Express, Gratuita }`. Functioneaza. Acum raspunde: daca maine apare `LivrareInternationala`, cate locuri din cod atingi? Ce reguli din open/closed incalci?
- Extrage fiecare ramura intr-o clasa-strategie.** Cele 3 clase din tabel implementeaza `ILivrareStrategie`. Fiecare tine DOAR formula ei — nimeni nu mai stie de celelalte.
- Injecteaza si schimba la runtime.** `Comanda` primeste o strategie prin constructor si are `SchimbaStrategie(ILivrareStrategie noua)`. `CostTransport()` delega strategiei curente — fara niciun `if / switch` pe tip in `Comanda`.

Cerinte

- Interfata `ILivrareStrategie` + cele 3 strategii.
- `Comanda` cu: `Client`, `GreutateKg`, `DistantaKm`, o strategie injectata, `SchimbaStrategie(...)` si `CostTransport()`. Constructorul valideaza: greutate `<= 0` sau distanta `< 0` arunca `ArgumentException` (mesaj in engleza).
- `Testare1` (chemata din `Program.cs`, ca la interfaces): o singura `Comanda`; calculeaza costul cu `LivrareStandard`, apoi `SchimbaStrategie` pe `LivrareExpress`, apoi pe `LivrareGratuita`; dupa fiecare afiseaza numele strategiei si costul. In output trebuie sa se vada ACEEASI comanda cu 3 costuri diferite.
- Demonstreaza validarea: incearca sa construiesti o comanda cu greutate `0`, prinde `ArgumentException` si afiseaza mesajul.

Constrangeri

- `Comanda` NU are voie sa contina `switch` / `if` pe tipul de livrare — toata logica de tip traieste in strategii.
- Fara `is` , `as` , `GetType()` . Fara LINQ, `List` , `Dictionary` — nu-ti trebuie aici.
- Mesajele de eroare in engleza.

Cum rulezi

```
dotnet run
```

Bonus (privire inainte spre Decorator, lectia 4)

- Scrie o strategie `LivrareCuReducere` care primeste in constructor O ALTA `ILivrareStrategie` si un procent, si returneaza costul strategiei interioare minus procentul. Observa: o strategie care impacheteaza alta strategie... exact asta e un **Decorator**. Il vom numi asa peste 3 lectii — deocamdata bucura-te ca Strategy ti l-a aratat gratis.
- Intrebare de gandit: de ce `CalculeazaCost` primeste `greutate` si `distanța` ca PARAMETRI, si nu ii tine strategia ca stare in campuri? Ce s-ar strica daca ai refolosi aceeasi instanta `LivrareStandard` pentru doua comenzi diferite, cu greutati diferite?



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro